

fastml: An R package for leakage-aware machine learning workflows with fold-local preprocessing

Selcuk Korkmaz¹, Dincer Goksuluk², and Eda Karaismailoglu³

¹ Department of Biostatistics, Trakya University, Edirne, Turkey ² Department of Biostatistics, Sakarya University, Sakarya, Turkey ³ Department of Medical Informatics, University of Health Sciences, Istanbul, Turkey ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- Review
- Repository
- Archive

Editor:

Submitted: 18 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Preprocessing leakage happens when a data-dependent transformation, such as scaling, imputation, class rebalancing or feature construction, is estimated on the whole dataset before resampling. Each fold's held-out observations have then already influenced the data the model trains on, and the performance estimate that follows is optimistic. This happens even for transformations that never touch the outcome labels (Kaufman et al., 2012; Moscovich & Rosset, 2022). One survey found the problem across seventeen scientific fields and in 294 papers (Kapoor & Narayanan, 2023), and in at least one domain it has been shown to inflate published results (Rosenblatt et al., 2024).

fastml is an R package that trains, tunes and compares machine-learning models through a single call. In fastml, *fold-local* preprocessing is the default execution path and not a convention the user has to remember to follow. Every data-dependent transformation is re-estimated inside each resampling split from that split's analysis data alone, then applied to the corresponding assessment data. The package covers classification, regression and survival tasks, and it offers resampling designs that respect grouping and time order alongside ordinary cross-validation. The evaluation design stays explicit and under the user's control.

Statement of need

Modular machine-learning frameworks break modelling into resampling, preprocessing, model specification and evaluation. Modularity is useful, but it also puts the burden of correct composition on the user, who has to apply the steps in the right order and keep training and assessment data strictly separated. Experienced practitioners still get this wrong in grouped, nested or time-ordered designs, and no software check catches it. A leaky pipeline is a perfectly valid pipeline: it runs, it produces a number, and nothing in the output suggests that anything is wrong.

We wrote fastml for applied researchers who need a defensible performance estimate more than they need a maximally flexible pipeline. For that audience a silent error is expensive, and the freedom to compose a pipeline in any order is worth little. Every framework discussed below can express a safe workflow. The harder question is how easily each one can also express an unsafe workflow by accident.

fastml answers that by narrowing the interface. Its preprocessing arguments compile to untrained specifications that are estimated only inside the resampling loop. A recipe already processed with `prep()` may contain parameters estimated outside the training process. The package therefore rejects it before training begins. We accepted this trade-off deliberately. The restricted interface prevents unsafe combinations but also rules out arbitrary pipeline structures, preprocessing parameter tuning, and stacking. Two limitations remain. The package cannot

undo preprocessing performed before the call. Also, a custom step in a user-supplied recipe can still access values computed elsewhere.

State of the field

Four R frameworks are the obvious points of comparison. `caret` (Kuhn, 2008) offers a legacy unified training interface. `tidymodels` (Kuhn & Wickham, 2020) emphasises explicit composition of recipes, model specifications, workflows and resampling. `mlr3` (Lang et al., 2019) provides an object-oriented task and learner design with a strong benchmarking culture, and `h2o` (Fryda et al., 2020) supplies an integrated AutoML system with its own training engine.

The comparison that matters most is with `mlr3pipelines` (Binder et al., 2021), because it couples preprocessing to evaluation in much the same way. Its `GraphLearner` composes preprocessing and a learner into a single object that is re-fitted on the training part of every fold, so fold-local preprocessing is already structural there and no separate guard is needed. `tidymodels` and `mlr3` can express leakage-safe workflows too. They can also express leakage-prone ones, just as easily and with no diagnostic.

`fastml` was developed not because existing frameworks cannot support safe workflows, but because their use often depends on user configuration and expertise. Its main distinctions are the default workflow and the constraints it enforces. The package restricts supported pipeline structures to ensure that preprocessing remains within the resampling loop. It rejects unsupported designs rather than executing an approximation that may compromise their validity. Survival modelling requires a more qualified comparison. Specialised frameworks such as `mlr3proba` (Sonabend et al., 2021) offer comparable or broader capabilities. In this setting, the contribution of `fastml` lies primarily in providing a consistent interface across heterogeneous learners, rather than greater methodological depth.

Software design

Under the guarded path, `fastml` works through each fold k in turn (Figure 1). It builds the split, fits a fresh preprocessing specification on the analysis data $D_{\text{analysis}}^{(k)}$ alone, applies that fold-trained specification to both $D_{\text{analysis}}^{(k)}$ and $D_{\text{assess}}^{(k)}$, and only then fits and evaluates the model. The assessment set is never used to estimate the transformation, only to be transformed by it. Tuning runs inside the same loop, so hyperparameter selection inherits the separation instead of undermining it.

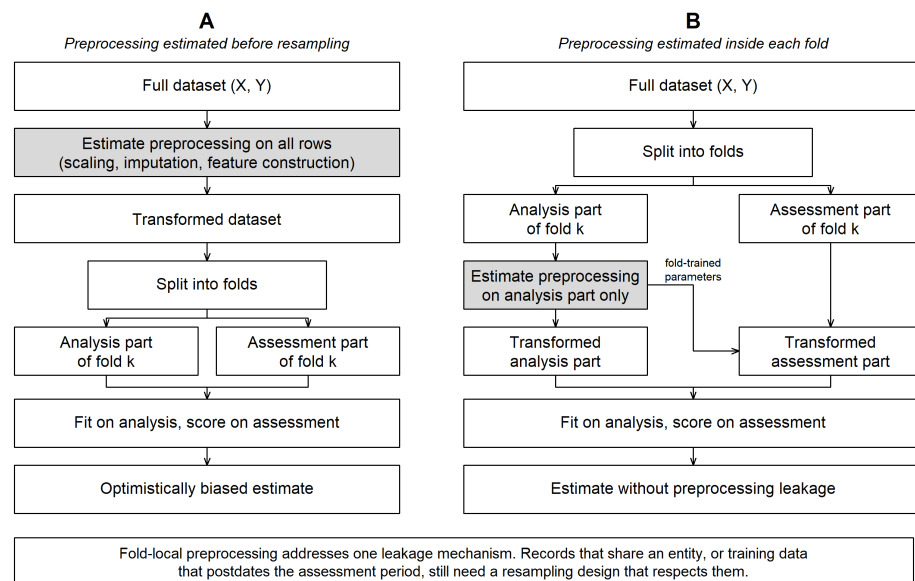


Figure 1: Where preprocessing is estimated. **(A)** Estimating a data-dependent transformation on the whole dataset before resampling lets each fold's assessment observations influence the data the model trains on, so the resulting score is optimistic. **(B)** The guarded path that `fastml` follows. The transformation is re-estimated from each fold's analysis part alone and then applied to both parts, so the assessment part is only ever transformed, never used to estimate the transformation itself.

The second design consideration concerned which requests the software should reject. Fold-local preprocessing addresses one source of leakage, but valid evaluation also requires a resampling design that accounts for dependencies in the data. For example, correlated observations from the same entity may be assigned to different folds (Roberts et al., 2017), or training observations may occur after the assessment period (Bergmeir & Benítez, 2012). `fastml` supports grouped, blocked, and rolling resampling designs to address these settings. It returns an error when a requested design cannot be executed as specified. In particular, execution stops if `blocked_cv` is used without an ordering column, `grouped_cv` is used without grouping columns, or a design requiring ordered data receives observations that are not sorted in ascending order of the ordering variable.

The initial holdout split follows the same principles. When data and `test_size` are supplied together, the split is determined by the user-specified dependence structure. If grouping columns are provided, entire groups are assigned to the holdout set. If an ordering column is provided, the final proportion of observations in temporal order is held out. Otherwise, the package uses a stratified row-wise split. This ensures that grouped cross-validation is not combined with a row-wise holdout.

Core functionality

A single `fastml()` call trains and compares models from 15 classification, 14 regression, and 11 survival model families using established engines within the `tidymodels` ecosystem. The package supports seven resampling designs: ordinary and repeated cross-validation, bootstrap, `grouped_cv`, `blocked_cv`, `rolling_origin`, and `nested_cv`. The `none` option provides holdout evaluation without resampling. Grouped, blocked, and rolling designs accommodate dependent data, whereas nested cross-validation separates hyperparameter selection from performance estimation. This separation reduces the selection bias that arises when the same folds are used

97 for both purposes (Varma & Simon, 2006).

98 Survival modelling is supported through both parsnip-compatible engines and native
99 implementations. These include the XGBoost accelerated failure time model with interval
100 bounds (Barnwal et al., 2020) and a piecewise-exponential model implemented through
101 flexsurv (Jackson, 2016). The package also provides utilities to screen user-supplied recipes
102 for dependencies on external data and the global environment. Additional features include
103 exploratory diagnostics and model interpretation using permutation importance, accumulated
104 local effects (ALE), LIME, individual conditional expectation (ICE) curves, surrogate trees,
105 and counterfactual explanations.

106 Example workflow

```
library(fastml)

fit <- fastml(
  data      = my_data,
  label     = "outcome",
  algorithms = c("rand_forest", "logistic_reg"),
  resampling_method = "cv",
  folds     = 10
)

summary(fit) # cross-validated comparison and the selected model
plot(fit, type = "roc")
predict(fit, newdata = new_rows)
```

107 Preprocessing options, including impute_method, scaling_methods, and balance_method, are
108 specified in the same call. Each option defines an untrained preprocessing step that is estimated
109 using only the training portion of each fold. Data dependencies are also specified within this
110 call: group_cols identifies grouping variables, while block_col defines the ordering variable
111 for blocked_cv or rolling_origin. These settings determine both the initial holdout split
112 and the resampling folds.

113 Research impact statement

114 fastml has been available on CRAN since November 2024 (Korkmaz et al., 2026a), with
115 approximately 108,700 downloads. Over 22 months of public development, the package has
116 had 15 CRAN releases, and its repository has received 109 stars and 8 forks.

117 Feedback from users outside the development group has informed package development.
118 Public issue reports include feedback on the handling of future backends from a maintainer of
119 widely used R parallelisation infrastructure, as well as reports from other users on visualisation
120 behaviour. A separate methodological manuscript uses Monte Carlo simulations and applied
121 case studies to quantify performance inflation attributable to preprocessing placement (Korkmaz
122 et al., 2026b). That manuscript presents the empirical evaluation, while the present article
123 focuses on the software.

124 AI usage disclosure

125 We used Anthropic's Claude models through Claude Code during the development of fastml
126 and the preparation of this manuscript, most recently Claude Opus 5. For software development,
127 AI assistance supported defect identification, diagnosis, and the drafting of code and tests.
128 The authors wrote the manuscript and used AI solely to improve language clarity and concision.

129 All package design decisions were made by the authors. Every AI-assisted software change
130 was reviewed, tested, and approved before inclusion, and all AI-assisted manuscript edits were
131 checked by the authors. The authors take full responsibility for the software and the manuscript,
132 including any remaining errors.

133 Conflicts of interest and funding

134 We declare no financial conflicts of interest. The work received no external funding.

135 Acknowledgements

136 We thank the users who reported issues against the package. Their feedback shaped its
137 interface and its resampling behaviour.

138 References

- 139 Barnwal, A., Cho, H., & Hocking, T. D. (2020). *Survival regression with accelerated failure*
140 *time model in XGBoost*. <https://doi.org/10.48550/arXiv.2006.04920>
- 141 Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor
142 evaluation. *Information Sciences*, 191, 192–213. <https://doi.org/10.1016/j.ins.2011.12.028>
- 143 Binder, M., Pfisterer, F., Lang, M., Schneider, L., Kotthoff, L., & Bischl, B. (2021).
144 mlr3pipelines – flexible machine learning pipelines in R. *Journal of Machine Learning*
145 *Research*, 22(184), 1–7.
- 146 Fryda, T., LeDell, E., Gill, N., Aiello, S., Fu, A., Candel, A., Click, C., Kraljevic, T., Nykodym,
147 T., Aboyoun, P., Kurka, M., & Malohlava, M. (2020). *h2o: R interface for the H2O*
148 *scalable machine learning platform*. <https://CRAN.R-project.org/package=h2o>
- 149 Jackson, C. (2016). flexsurv: A platform for parametric survival modeling in R. *Journal of*
150 *Statistical Software*, 70(8), 1–33. <https://doi.org/10.18637/jss.v070.i08>
- 151 Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-
152 based science. *Patterns*, 4(9), 100804. <https://doi.org/10.1016/j.patter.2023.100804>
- 153 Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining:
154 Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from*
155 *Data*, 6(4). <https://doi.org/10.1145/2382577.2382579>
- 156 Korkmaz, S., Goksuluk, D., & Karaismailoglu, E. (2026a). *fastml: Guarded resampling*
157 *workflows for leakage-aware machine learning*. [https://doi.org/10.32614/CRAN.package.](https://doi.org/10.32614/CRAN.package.fastml)
158 [fastml](https://doi.org/10.32614/CRAN.package.fastml)
- 159 Korkmaz, S., Goksuluk, D., & Karaismailoglu, E. (2026b). *fastml: Guarded resampling*
160 *workflows for safer automated machine learning in R*. [https://doi.org/10.48550/arXiv.2604.](https://doi.org/10.48550/arXiv.2604.05225)
161 [05225](https://doi.org/10.48550/arXiv.2604.05225)
- 162 Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of*
163 *Statistical Software*, 28(5), 1–26. <https://doi.org/10.18637/jss.v028.i05>
- 164 Kuhn, M., & Wickham, H. (2020). *Tidymodels: A collection of packages for modeling and*
165 *machine learning using tidyverse principles*. <https://www.tidymodels.org>
- 166 Lang, M., Binder, M., Richter, J., Schratz, P., Pfisterer, F., Coors, S., Au, Q., Casalicchio,
167 G., Kotthoff, L., & Bischl, B. (2019). mlr3: A modern object-oriented machine learning
168 framework in R. *Journal of Open Source Software*, 4(44), 1903. <https://doi.org/10.21105/joss.01903>
169

- 170 Moscovich, A., & Rosset, S. (2022). On the cross-validation bias due to unsupervised
171 preprocessing. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*,
172 84(4), 1474–1502. <https://doi.org/10.1111/rssb.12537>
- 173 Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Aroita, G., Hauenstein,
174 S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig,
175 F., & Dormann, C. F. (2017). Cross-validation strategies for data with temporal, spatial,
176 hierarchical, or phylogenetic structure. *Ecography*, 40(8), 913–929. <https://doi.org/10.1111/ecog.02881>
177
- 178 Rosenblatt, M., Tejavibulya, L., Jiang, R., Noble, S., & Scheinost, D. (2024). Data leakage
179 inflates prediction performance in connectome-based machine learning models. *Nature*
180 *Communications*, 15(1). <https://doi.org/10.1038/s41467-024-46150-w>
- 181 Sonabend, R., Király, F. J., Bender, A., Bischl, B., & Lang, M. (2021). mlr3proba: An R
182 package for machine learning in survival analysis. *Bioinformatics*, 37(17), 2789–2791.
183 <https://doi.org/10.1093/bioinformatics/btab039>
- 184 Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model
185 selection. *BMC Bioinformatics*, 7(1), 91. <https://doi.org/10.1186/1471-2105-7-91>

DRAFT